

Python & Pandas Setup Guide

Windows + VS Code + Git Bash

Modern Data Engineering Course — Week 2

Step 1: Make Sure You Have Git Bash

Git Bash is a terminal for Windows that gives you a Linux-like command line. Check if you already have it:

- Press the **Windows key**, type "**Git Bash**", and see if it appears
- Or look in: `C:\Program Files\Git\git-bash.exe`

If you don't have Git Bash:

1. Go to <https://git-scm.com/downloads/win>
2. Download the installer (64-bit)
3. Run it — keep all the default options, just click Next through everything
4. After installation, search "**Git Bash**" in Start menu — it should now appear

💡 **Why Git Bash?** It gives you a proper terminal on Windows (similar to Mac/Linux). We'll use Git itself later in Session 12 for version control, so you need it anyway.

Step 2: Install Python 3.12

Open **Git Bash** and run:

```
winget install Python.Python.3.12
```

Close and reopen Git Bash, then verify:

```
py -3.12 --version
```

You should see `Python 3.12.x`. If `winget` doesn't work, download the installer manually from <https://www.python.org/downloads/release/python-3129/> — during installation, check **"Add Python to PATH"**.

 **Why 3.12?** Python 3.13 and 3.14 are too new — many data libraries don't support them yet. 3.12 is the safe choice.

Step 3: Install VS Code Extensions

Open VS Code, press `Ctrl+Shift+X`, and install:

1. **Python** (by Microsoft)
 2. **Jupyter** (by Microsoft)
-

Step 4: Set Up Your Project

4.1 — Create your project folder and open it in VS Code

File → Open Folder → pick your lesson folder

4.2 — Open Git Bash terminal inside VS Code

Terminal → New Terminal

Make sure the terminal is **Git Bash**, not PowerShell. If it shows PowerShell, click the dropdown arrow (v) next to the + button in the terminal panel and select **Git Bash**.

4.3 — Create a virtual environment

```
py -3.12 -m venv pandas_venv
```

4.4 — Activate it

```
source pandas_venv/Scripts/activate
```

You should see `(pandas_venv)` at the start of your prompt. This means you're inside the virtual environment.

4.5 — Install packages

```
pip install pandas==2.2.3 numpy pyarrow==17.0.0 openpyxl ipykernel
```

4.6 — Verify

```
python -c "import pandas; import pyarrow; print(f'Pandas {pandas.__version__}, PyArrow {pyarrow.__version__}')
```

You should see: **Pandas 2.2.3, PyArrow 17.0.0**

Step 5: Open a Notebook & Select the Kernel

1. In VS Code: **Ctrl+Shift+P** → type **"Jupyter: Create New Blank Notebook"** → Enter
2. Look at the **top right corner** — click **"Select Kernel"**
3. Click **"Python Environments..."**
4. Pick the one that shows **pandas_venv**, something like **Python 3.12.x ('pandas_venv')**

 **Don't see it?** Close VS Code completely and reopen it. Then try again.

Test it — run this in the first cell:

```
import pandas as pd
```

```
import numpy as np
```

```
print(f"✅ Pandas {pd.__version__}")
```

```
print(f"✅ NumPy {np.__version__}")
```

```
df = pd.DataFrame({"name": ["Rahim", "Fatima"], "score": [85, 92]})
```

```
df.to_parquet("test.parquet")
```

```
print("✅ Parquet works!")
```

Three  = you're all set!

Daily Workflow

Every time you open your project:

1. Open VS Code → Open your project folder
 2. Open terminal (Git Bash) → `source pandas_venv/Scripts/activate`
 3. Open your notebook → check the kernel (top right) shows `pandas_venv`
 4. Code!
-

Troubleshooting

Problem	Fix
<code>python is not recognized</code>	Reinstall Python, check "Add to PATH"
<code>ModuleNotFoundError: No module named 'pandas'</code>	Wrong kernel selected — switch to <code>pandas_venv</code> (Step 5)
Parquet gives <code>ArrowKeyError</code>	Run <code>pip install pandas==2.2.3 pyarrow==17.0.0 --force-reinstall</code> then restart the kernel (🔄 button)
Don't see <code>pandas_venv</code> in kernel list	Close and reopen VS Code
Terminal shows PowerShell not Git Bash	Click dropdown (v) next to + in terminal → select Git Bash

Appendix: requirements.txt

Instead of typing package names manually, create a file called `requirements.txt` in your project folder:

```
pandas==2.2.3
```

```
numpy
```

```
pyarrow==17.0.0
```

openpyxl

ipykernel

Then install everything with one command:

```
pip install -r requirements.txt
```

This is how real projects manage dependencies. You'll use this pattern throughout the course — especially when we get to Docker (Session 13) and CI/CD pipelines (Session 25).

Setup done! You're ready for Week 2. 🚀